

Experiment-01 Home Tasks

Tasks:

A. [Experiment - 1C] A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called `pizza_toppings`. Every customer must choose exactly 3 different toppings to prepare a pizza. Write a PostgreSQL query to generate every possible 3-topping pizza combination along with its total ingredient cost.

Requirements:

- Every pizza must contain exactly 3 different toppings.
- The same topping cannot appear more than once in a pizza.
- Different arrangements of the same toppings should be considered the same pizza.
Example: Chicken, Pepperoni, Sausage / Pepperoni, Chicken, Sausage / Sausage, Chicken, Pepperoni — these are all the same pizza, so only one should appear.
- Display the topping names separated by commas.
- Display the total ingredient cost of each pizza.
- Sort the output: first by highest total cost; if tied, sort alphabetically.

B. [Experiment – 1D] Amazon wants to identify returning customers who made another purchase shortly after their first purchase. Write a PostgreSQL query to find all users who made another purchase within 1 to 7 days of an earlier purchase.

Requirements:

- Compare purchases made by the same user only.
- Ignore comparisons where the same transaction is matched with itself.
- The second purchase must occur after the first purchase.
- Same-day purchases must not be considered.
- The second purchase must occur within 7 days of the first purchase.
- Display only the unique User IDs that satisfy the above conditions.
- Sort the output in ascending order of `user_id`.

Tools / Platform Used:

- Windows
- PostgreSQL
- pgAdmin

Code and Output:

A.

```
CREATE TABLE pizza_toppings (  
    topping_name VARCHAR(50) PRIMARY KEY,  
    ingredient_cost NUMERIC(4,2) NOT NULL  
);  
  
INSERT INTO pizza_toppings (topping_name, ingredient_cost) VALUES  
( 'Mushroom', 0.45),  
( 'Paneer', 0.65),  
( 'Corn', 0.30),  
( 'Capsicum', 0.35),  
( 'Jalapeno', 0.50);
```

```
CREATE TABLE pizza_toppings (  
    topping_name VARCHAR(50) PRIMARY KEY,  
    ingredient_cost NUMERIC(4,2) NOT NULL  
);  
  
INSERT INTO pizza_toppings (topping_name, ingredient_cost) VALUES  
( 'Mushroom', 0.45),  
( 'Paneer', 0.65),  
( 'Corn', 0.30),  
( 'Capsicum', 0.35),  
( 'Jalapeno', 0.50);
```

STDIN

Input for the program (Optional
)

Output:

CREATE TABLE
INSERT 0 5

```
SELECT  
    t1.topping_name || ', ' || t2.topping_name || ', ' || t3.topping_name AS  
pizza,  
    ROUND(  
        t1.ingredient_cost + t2.ingredient_cost + t3.ingredient_cost,  
        2  
    ) AS total_cost  
FROM pizza_toppings t1  
JOIN pizza_toppings t2  
    ON t1.topping_name < t2.topping_name  
JOIN pizza_toppings t3  
    ON t2.topping_name < t3.topping_name  
ORDER BY  
    total_cost DESC,  
    pizza ASC;
```

```

1 CREATE TABLE pizza_toppings (
2     topping_name VARCHAR(50) PRIMARY KEY,
3     ingredient_cost NUMERIC(4,2) NOT NULL
4 );
5
6 INSERT INTO pizza_toppings (topping_name, ingredient_cost) VALUES
7 ('Mushroom', 0.45),
8 ('Paneer', 0.65),
9 ('Corn', 0.30),
0 ('Capsicum', 0.35),
1 ('Jalapeno', 0.50);
2
3 SELECT
4     t1.topping_name || ', ' || t2.topping_name || ', ' || t3.topping_name AS pizza,
5     ROUND(
6         t1.ingredient_cost + t2.ingredient_cost + t3.ingredient_cost,
7         2
8     ) AS total_cost
9 FROM pizza_toppings t1
10 JOIN pizza_toppings t2
11     ON t1.topping_name < t2.topping_name
12 JOIN pizza_toppings t3
13     ON t2.topping_name < t3.topping_name
14 ORDER BY
15     total_cost DESC,
16     pizza ASC;

```

STDIN

Input for the program (Optional)

Output:

CREATE TABLE

INSERT 0 5

pizza	total_cost
Jalapeno, Mushroom, Paneer	1.60
Capsicum, Jalapeno, Paneer	1.50
Capsicum, Mushroom, Paneer	1.45
Corn, Jalapeno, Paneer	1.45
Corn, Mushroom, Paneer	1.40
Capsicum, Corn, Paneer	1.30
Capsicum, Jalapeno, Mushroom	1.30
Corn, Jalapeno, Mushroom	1.25
Capsicum, Corn, Jalapeno	1.15
Capsicum, Corn, Mushroom	1.10

(10 rows)

B.

```

CREATE TABLE amazon_transactions (
    id INT PRIMARY KEY,
    user_id INT NOT NULL,
    item VARCHAR(50),
    created_at TIMESTAMP NOT NULL,
    revenue INT
);

```

```

INSERT INTO amazon_transactions (id, user_id, item, created_at, revenue) VALUES
(1, 201, 'notebook', '2026-02-01 09:00:00', 10),
(2, 201, 'pen', '2026-02-01 15:00:00', 3),
(3, 201, 'eraser', '2026-02-06 10:00:00', 2),
(4, 202, 'charger', '2026-02-10 11:00:00', 15),
(5, 202, 'cable', '2026-02-28 12:00:00', 6),
(6, 203, 'headphones', '2026-02-12 08:00:00', 20),
(7, 203, 'case', '2026-02-14 09:30:00', 8);

```

```

CREATE TABLE amazon_transactions (
    id INT PRIMARY KEY,
    user_id INT NOT NULL,
    item VARCHAR(50),
    created_at TIMESTAMP NOT NULL,
    revenue INT
);

```

```

INSERT INTO amazon_transactions (id, user_id, item, created_at, revenue) VALUES
(1, 201, 'notebook', '2026-02-01 09:00:00', 10),
(2, 201, 'pen', '2026-02-01 15:00:00', 3),
(3, 201, 'eraser', '2026-02-06 10:00:00', 2),
(4, 202, 'charger', '2026-02-10 11:00:00', 15),
(5, 202, 'cable', '2026-02-28 12:00:00', 6),
(6, 203, 'headphones', '2026-02-12 08:00:00', 20),
(7, 203, 'case', '2026-02-14 09:30:00', 8);

```

STDIN

Input for the program (Optional)

Output:

CREATE TABLE

INSERT 0 7

```

SELECT DISTINCT t1.user_id
FROM amazon_transactions t1
JOIN amazon_transactions t2
    ON t1.user_id = t2.user_id
    AND t1.id <> t2.id
    AND t2.created_at > t1.created_at
    AND t2.created_at::date > t1.created_at::date

```

```
AND t2.created_at <= t1.created_at + INTERVAL '7 days'
ORDER BY t1.user_id;
```

```
1 CREATE TABLE amazon_transactions (
2     id INT PRIMARY KEY,
3     user_id INT NOT NULL,
4     item VARCHAR(50),
5     created_at TIMESTAMP NOT NULL,
6     revenue INT
7 );
8
9 INSERT INTO amazon_transactions (id, user_id, item, created_at, revenue) VALUES
10 (1, 201, 'notebook', '2026-02-01 09:00:00', 10),
11 (2, 201, 'pen', '2026-02-01 15:00:00', 3),
12 (3, 201, 'eraser', '2026-02-06 10:00:00', 2),
13 (4, 202, 'charger', '2026-02-10 11:00:00', 15),
14 (5, 202, 'cable', '2026-02-28 12:00:00', 6),
15 (6, 203, 'headphones', '2026-02-12 08:00:00', 20),
16 (7, 203, 'case', '2026-02-14 09:30:00', 8);
17 SELECT DISTINCT t1.user_id
18 FROM amazon_transactions t1
19 JOIN amazon_transactions t2
20     ON t1.user_id = t2.user_id
21     AND t1.id <> t2.id
22     AND t2.created_at > t1.created_at
23     AND t2.created_at::date > t1.created_at::date
24     AND t2.created_at <= t1.created_at + INTERVAL '7 days'
25 ORDER BY t1.user_id;
```

SQLIN

Input for the program (Optional)

Output:

CREATE TABLE

INSERT 0 7

user_id

201

203

(2 rows)

Result:

The queries for the given tasks were written and executed successfully. The tasks given were completed successfully.

Learning Outcomes:

- Practiced PostgreSQL
- Got hands-on experience on working with JOINS in PostgreSQL.
- Understood the concept of JOINS.